

job-genie.ai — Canonical & Masking Fix Runbook

Goal: kill GoDaddy masked forwarding, serve the site directly from Replit on both hosts, and 301 the apex to `www` so search engines and AI crawlers see one canonical URL per page.

Chosen canonical host: `https://www.job-genie.ai` (matches the `canonical` tags already on your pages — no need to rewrite 45+ tags).

⚠ **Order matters.** Do Part A first. While GoDaddy forwarding/masking is on, requests never reach your Replit app, so the app-level 301 in Part C can't run.

Part A — GoDaddy: kill the masking (root cause)

The bare domain is being served through GoDaddy **Domain Forwarding with masking** (the signature: `job-genie.ai` returns a `200` with a frame wrapper instead of a `301`). This traps your content behind a frame and is why almost nothing is indexed.

1. GoDaddy → **My Products** → your domain → **Domain Settings**.
2. Find the **Forwarding** section (Domain + Subdomain).
3. **Delete** any forwarding rule — both the apex (`job-genie.ai`) rule and any `www` or `app` rule. Do **not** just disable "masking"; remove the forwarding entirely.
4. If you were masked-forwarding the apex to `app.job-genie.ai`, that forward *is* the masking — delete it. The blog should be served directly by the linked Replit deployment, not via a forward.

If `app.job-genie.ai` is a genuinely separate application (dashboard) that you want to keep, don't fold it into this. Only the **marketing/blog** host needs to resolve directly to the deployment. (Flag this to me and I'll split the config.)

Part B — Replit + GoDaddy DNS: point both hosts at the deployment

Replit generates a **per-domain A + TXT pair** — there is no fixed IP to reuse. You must link **two** domains because linking the apex does **not** make `www` resolve.

In Replit

1. Open your **Deployment** → **Settings** → **Link a domain**.
2. Link `www.job-genie.ai` → copy its **A record IP** and `replit-verify=...` **TXT** value.
3. Click **Link a domain** again, link `job-genie.ai` (apex) → copy *its* A + TXT (these are different values).

In GoDaddy DNS (after forwarding is deleted)

Remove any leftover GoDaddy **parking/forwarding A records** first (typically `@` and `www` pointing to GoDaddy/AWS forwarding IPs). Then add:

Type	Name	Value	Notes
A	@	(apex A IP from Replit)	replaces any parked/forwarding A record
TXT	@	replit-verify=... (apex)	must stay permanently — removing it breaks SSL renewal
A	www	(www A IP from Replit)	separate value from the apex
TXT	www	replit-verify=... (www)	keep permanently

Leave your existing **beehiv** subdomain records untouched.

Do not re-enable any forwarding. **Do not** turn on Cloudflare-style proxying if you add Cloudflare later — proxying can break Replit's automatic SSL renewal.

Replit auto-provisions and renews SSL for each linked domain via DNS validation. Propagation is usually minutes but can take up to 48h.

Part C — App-level 301 (apex + http → https://www)

Both hosts now serve 200. The app must permanently redirect job-genie.ai → www.job-genie.ai so there's one canonical URL. Add this **before** your routes.

Express / Node (most likely for your stack)

```
js
```

```
// Trust Replit's proxy so host/proto are read correctly
app.set('trust proxy', 1);

const CANONICAL_HOST = 'www.job-genie.ai';

app.use((req, res, next) => {
  const host = (req.headers.host || '').toLowerCase();
  const proto = (req.headers['x-forwarded-proto'] || req.protocol || 'http').toLowerCase();

  const isOurDomain = host === 'job-genie.ai' || host === CANONICAL_HOST;

  // Force apex → www and http → https, but leave *.replit.app preview and localhost
  if (isOurDomain && (host !== CANONICAL_HOST || proto !== 'https')) {
    return res.redirect(301, `https://${CANONICAL_HOST}${req.originalUrl}`);
  }
  next();
});
```

This preserves the full path + query string, uses a permanent (301), and deliberately skips the (.replit.app) preview URL and (localhost) so you don't break dev or the Replit preview.

Next.js (only if the blog is a Next app)

```
js
// next.config.js
module.exports = {
  async redirects() {
    return [
      {
        source: '/*:path*',
        has: [{ type: 'host', value: 'job-genie.ai' }],
        destination: 'https://www.job-genie.ai/:path*',
        permanent: true, // emits 301
      },
    ];
  },
};
```

If the blog is a **Static** Replit deployment (no server), host-based 301s aren't possible there — tell me and we'll handle canonicalization a different way (e.g., link only [www](https://www.job-genie.ai) and don't link the apex).

Part D — Verify, then re-index

Run these after propagation:

```
bash

# Apex must now 301 to www (NOT return 200 with a frame)
curl -sI https://job-genie.ai/blog | grep -iE 'HTTP/|location'
# expect: HTTP/2 301 + location: https://www.job-genie.ai/blog

# www must serve the page
curl -sI https://www.job-genie.ai/blog | grep -iE 'HTTP/|location'
# expect: HTTP/2 200

# http must upgrade to https+www
curl -sI http://job-genie.ai/blog | grep -iE 'HTTP/|location'
# expect: 301 → https://www.job-genie.ai/blog
```

If the apex still returns `200` with no `location` header, the GoDaddy forward wasn't fully removed — go back to Part A.

Then:

1. **Google Search Console** → add/confirm the `www.job-genie.ai` property.
2. Confirm `https://www.job-genie.ai/sitemap.xml` exists, lists **all** posts as `www` URLs, and is referenced in `robots.txt` (`Sitemap: https://www.job-genie.ai/sitemap.xml`).
3. Submit the sitemap; use **URL Inspection** → **Request indexing** on the `/blog` index and a few key posts.
4. Add pagination or an archive link so older posts aren't orphaned once they roll off the index.

Why this fixes the AI-crawler problem

AI answer engines can only cite pages that are crawled, indexed, and unambiguously canonical. Right now you have masking (content trapped in a frame) **and** a split link graph (two live hosts contradicting their own canonical). Collapsing everything to a single `301`-enforced `www` host removes both blockers, consolidates any inbound link equity, and lets the sitemap actually drive indexation.